Meridian

Apartment Complex Management System

Team 98 · IITM Software Engineering · May 2026

> *Speaker notes are in italics under each slide. They are what to say, not what to put on screen — the slide itself should stay sparse.*

1. The problem

Apartment societies run on spreadsheets, WhatsApp groups and paper registers.

From our Milestone 1 user research:

- No visibility into **what is owed** or whether it was received
- Complaints raised **verbally**, with no trace and no accountability
- **Double-booked** facilities
- Visitors admitted on the strength of a **phone call**
- Owners living elsewhere with **no way to see any of it**

> *These are five separate failures with one shared cause: nothing is written down anywhere both sides can see. That is what we set out to fix.*

2. Users — 9 roles, 4 tiers

Tier	Role
Primary	Resident / Flat Owner · Society Official
Secondary	Non-Resident Owner · Visitor
Tertiary	Security Guard · Maintenance Staff · Cleaning Staff · Service Provider
Platform	Super Admin

Visitors are deliberately not accounts. They hold a signed, time-boxed, single-use pass token.

> *That decision is worth pausing on: it keeps the auth model simple — no visitor registration, no visitor login — while still giving the guard something cryptographically verifiable to check at the gate.*

3. Six epics, 24 user stories

Epic	Stories
E1 — Maintenance Management	4
E2 — Complaint Management	4
E3 — Facility Management	4
E4 — Visitor & Emergency Management	5
E5 — Society Administration	4
E6 — Non-Resident Owner Services	3

The stories are **embedded in the API contract**, not just listed in a document.

> *Every story names the endpoints that implement it, as an `x-user-story-mapping` extension in the published OpenAPI spec. So the traceability travels with the contract instead of rotting in a separate file.*

4. The decision that shaped everything

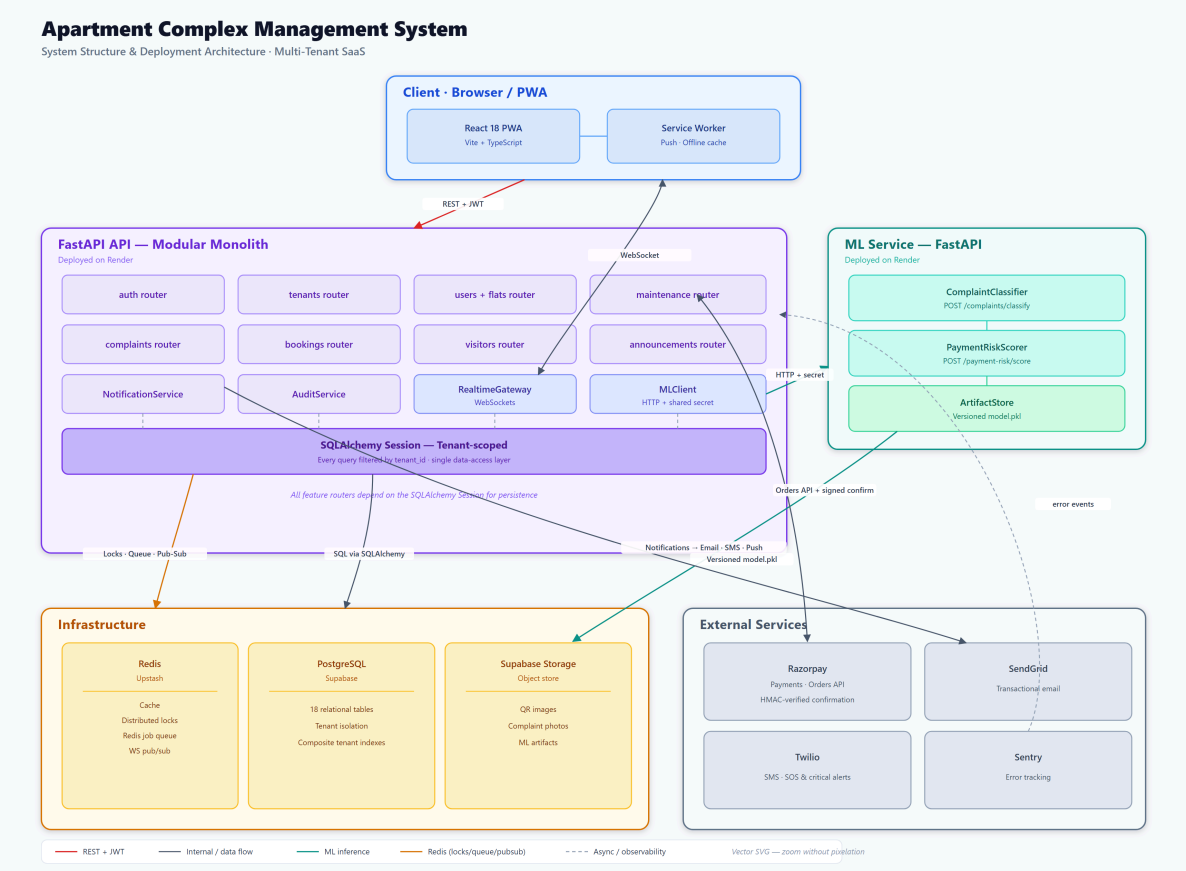
Multi-tenant SaaS, not single-society

One deployment serves many societies, each isolated by `tenant_id`.

- Scope comes from the **JWT claim** — never a request parameter
- Cross-tenant reads return **404, not 403**
- A dedicated test suite attacks **every** cross-society path with a foreign token

> *We went beyond the literal brief here. It is a materially harder engineering problem without much extra feature work, and it is the shape real PropTech products have. The 404-not-403 detail matters: a 403 would confirm the record exists, which leaks that another society holds that id.*

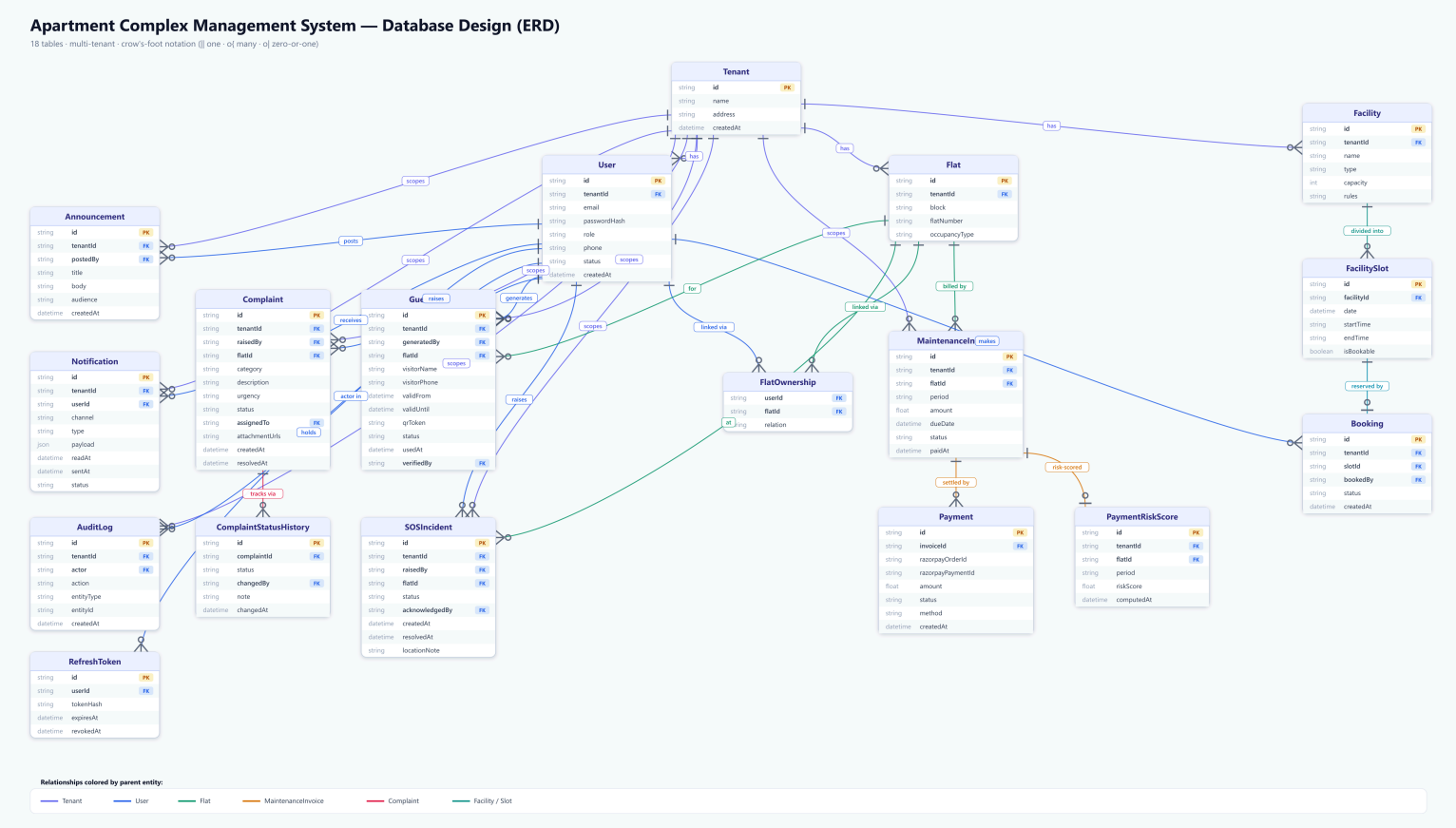
5. Architecture



Modular monolith for the API — one router per epic — plus a separately deployable ML service.

> *Not microservices. At five people, one router per epic gives the same separation without the operational cost of independent deploys. The ML service is the one exception, because one member owns both models and needs to ship them without touching API releases.*

6. Database — 18 tables



> *Every tenant-scoped table carries `tenant_id`, and every composite index leads with it. Enums are VARCHAR with a CHECK constraint rather than native Postgres enums — keeps values readable and avoids the migration cost of altering an enum.*

7. Technology

Layer	Choice
Frontend	React 18 · TypeScript · Vite · TanStack Query · Zustand · Tailwind
Backend	FastAPI · SQLAlchemy 2.0 · Alembic · PyJWT · PBKDF2-SHA256
Database	PostgreSQL (Supabase) · SQLite for local and test
ML	scikit-learn · TF-IDF + Logistic Regression · Gradient Boosting
External	Razorpay (test mode) · Google Gemini
Infra	Render · Vercel · Upstash Redis · GitHub Actions
Quality	pytest · Ruff · `tsc --noEmit`

> *One language across the API and the ML service, deliberately — same team, one toolchain, and the backend is directly testable with pytest.*

8. The API contract

48 paths · 59 operations · 62 schemas · 44 catalogued error codes

Generated from the live route table — it **cannot** describe an endpoint that does not exist.

CI enforces the converse: re-export the spec, fail on any diff.

```
{ "statusCode": 409, "error": "SLOT_ALREADY_BOOKED",
  "message": "This slot has already been booked", "requestId": "…" }
```

> *Every failure in the system has that shape.* `error` *is a stable code the frontend branches on;* `message` *is prose we can reword freely.* `requestId` *comes back in a header too, so a user-reported bug traces straight to its log line.*

9. Preventing the double-booking race

Milestone 1 named this explicitly. Defended **twice**:

1. A per-slot **lock** (Redis, or in-process) — the loser gets a clean `409`
2. A **partial unique index** on `slot_id WHERE status = 'CONFIRMED'` — makes a second confirmed booking *impossible*

Tested with **eight genuinely simultaneous threads**: exactly one `201`, seven `409`s.

> *The index is the real guarantee; the lock is an optimisation that produces a tidy error instead of a database exception. The index is partial so cancelling frees the slot again.*

10. Machine learning

Complaint classifier — TF-IDF (uni + bigrams) → Logistic Regression

Payment risk — Gradient-boosted trees over 5 features

Model	Metric
Category	macro-F1 1.0
Urgency	macro-F1 0.99
Payment risk	ROC-AUC 0.67 *(Bayes-optimal 0.68)*

Read these carefully — we do.

> *The 1.0 is not a generalisation estimate: train and test come from the same phrase templates, so it largely measures the templates. On phrasings written nowhere in the training set it got 4 of 6 — and returned confidence 0.30 and 0.39 on the misses, below the backend's threshold, so the fallback caught them. The test now asserts "never confidently wrong" rather than "never wrong". And 0.67 looks weak until you see the Bayes-optimal for that process is 0.68 — it is at 98% of achievable.*

11. ML never blocks a user action

If the ML service is **down, slow, or unsure** → deterministic keyword classifier, complaint flagged for manual triage.

The complaint is **always** created.

Same pattern everywhere:

Missing	Falls back to
ML service	Keyword classification
Gemini key	Rule-based intent matching
Razorpay keys	Official records payment manually
Redis	In-process lock and rate limiter

> *This is the single design principle we are proudest of. Every integration degrades to something that works, which is why the whole system can be demoed with almost nothing configured.*

12. Engineering quality

- **149 backend + 21 ML tests**, all passing
- **Tenant isolation** suite — every cross-society path, foreign token
- **Concurrency** — 8 real threads on one slot
- **Migration parity** — Alembic revisions vs ORM models compared structurally
- **Live-Redis** suite — a fake cannot prove `SET NX` is atomic
- Ruff · `tsc --noEmit` · 3 CI workflows · contract-drift gates

> *The migration test is the one we would point at. Two paths build the schema — `create_all` locally, Alembic on Postgres — and they can drift. It builds both and compares columns, foreign keys, indexes and constraints, so a model change without a revision fails in CI rather than on deploy.*

13. Testing found real defects

TC-11 — an expired guest pass was still listed as `ACTIVE`

Found by testing → documented with root cause → held under a strict `xfail` → scheduled → fixed → verified green.

And in the final sprint, the Redis lock tests found two more:

- `EVAL` unavailable → lock stranded until TTL, stalling every booking for that slot
- Contended lock and unreachable Redis handled identically → **two threads inside the critical section at once**

> *Neither of those would have surfaced against a fake. That is the argument for testing against a real service.*

14. Process

- **Jira** — SCRUM board, sprints, burndown
- **GitHub** — branch → PR → CI → review → merge
- **13 commits** carry their Jira issue id, so board items trace to code
- 56 commits · 9 branches · 4 contributors

> *User feedback changed the product: one item we accepted and built, one we declined with a reasoned alternative — an official wanted to skip the in-progress state, and we kept it because it is what makes the audit trail meaningful, then gave them a one-tap action that issues both transitions.*

15. Live demo

> *Switch to the application.*

16. What we would do next

1. **Frontend test suite** — Vitest, RTL, then Playwright. The largest genuine hole.
2. **Real notification delivery** — push, email, SMS behind the existing boundary
3. **WebSocket gateway** — replacing polling for SOS and slot availability
4. **ML on real data** — both models are trained on bootstrapped data today
5. Complaint photo upload · PWA · Sentry

> *We would rather state these than have someone find them. A disclosed gap is engineering judgement; an undisclosed one makes a reader wonder what else is undisclosed.*

Thank you

Questions?

Repository · Live demo · `DEPLOYMENT.md` · Final report